

Topic 13

Arithmetic Components

Components to be discussed

- Arithmetic and logic unit (ALU)
- Carry lookahead adder
- Multiplier

Arithmetic-Logic Unit: ALU

- **ALU**: Component that can perform any of various arithmetic (add, subtract, increment, etc.) and logic (AND, OR, etc.) operations, based on control inputs
- Key component in computer

TABLE 4.2 Desired calculator operations

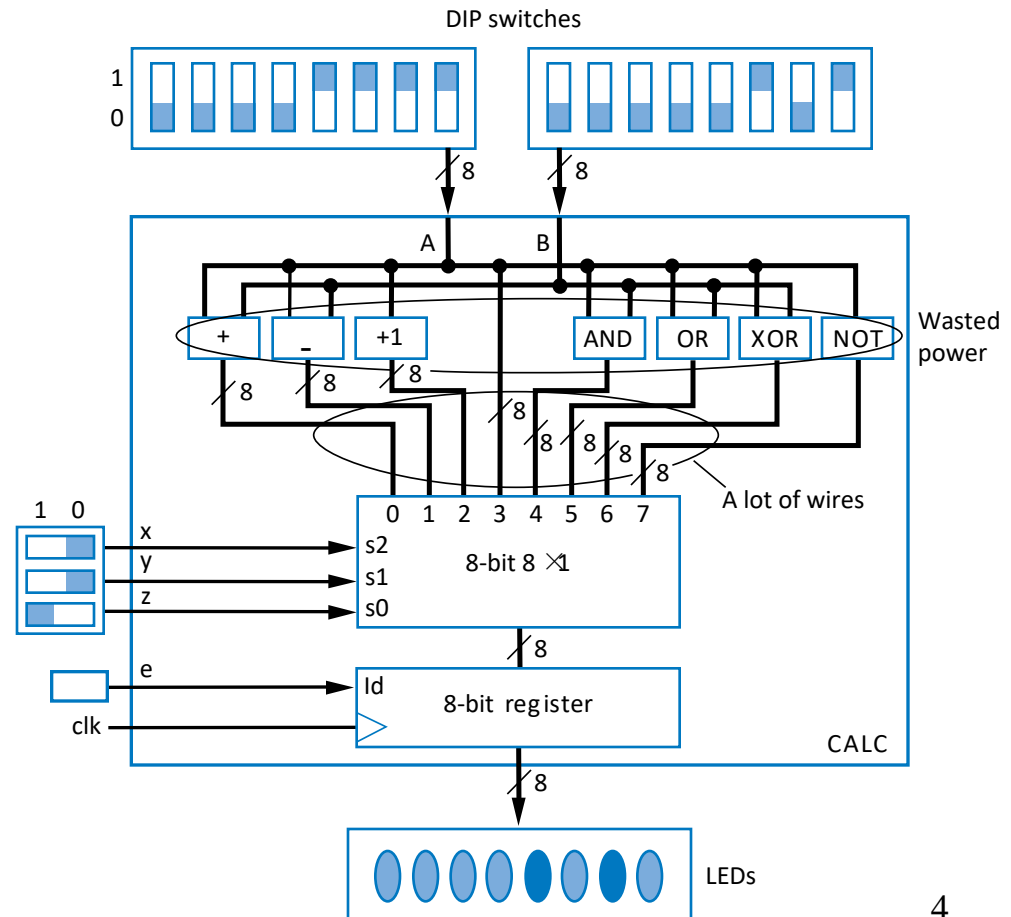
Inputs			Operation	Sample output if A=00001111, B=00000101
x	y	z		
0	0	0	$S = A + B$	S=00010100
0	0	1	$S = A - B$	S=00001010
0	1	0	$S = A + 1$	S=00010000
0	1	1	$S = A$	S=00001111
1	0	0	$S = A \text{ AND } B$ (bitwise AND)	S=00000101
1	0	1	$S = A \text{ OR } B$ (bitwise OR)	S=00001111
1	1	0	$S = A \text{ XOR } B$ (bitwise XOR)	S=00001010
1	1	1	$S = \text{NOT } A$ (bitwise complement)	S=11110000

Multifunction Calculator with an ALU

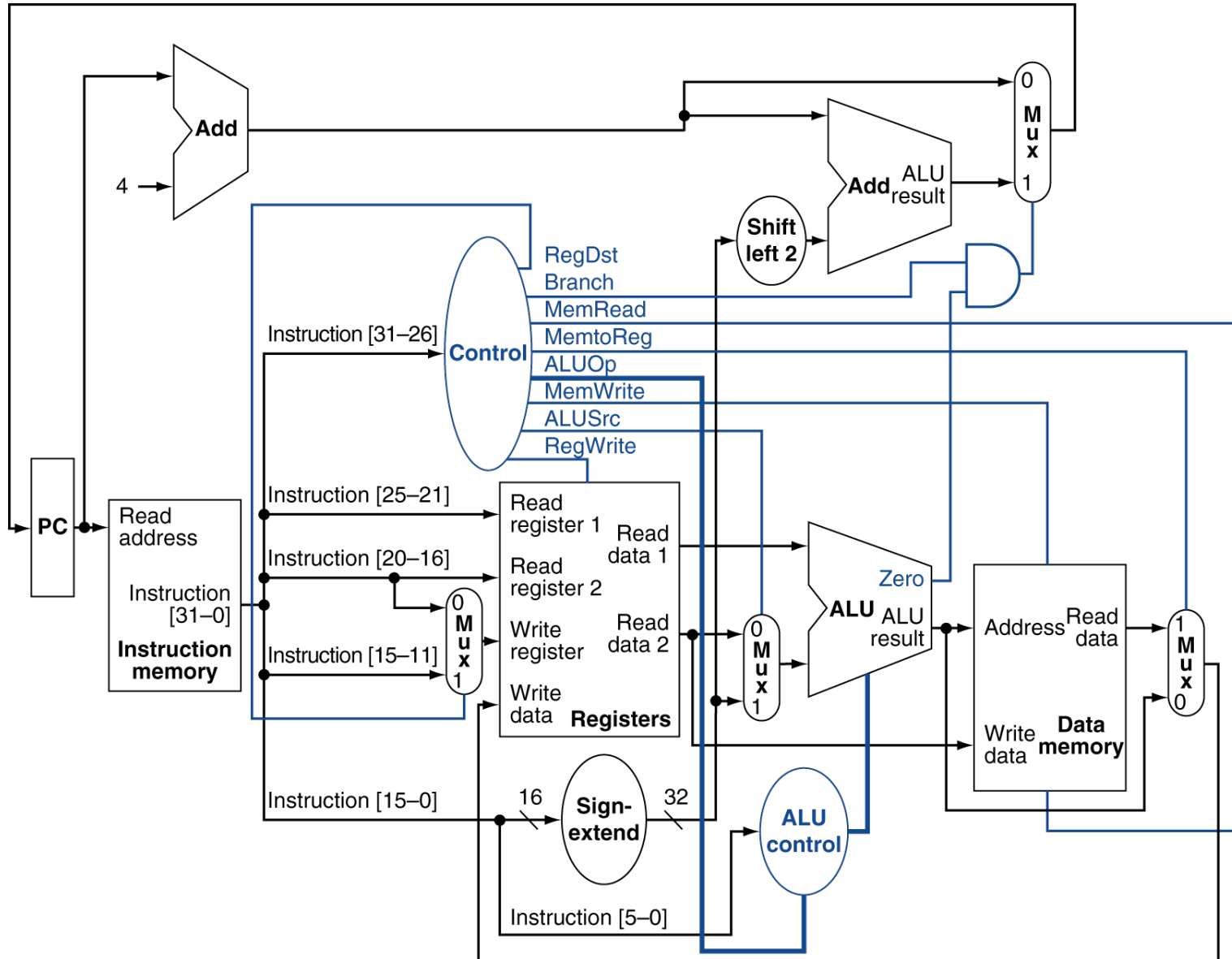
- ALU functions selected by a mux
 - But too many wires
 - Wasted power computing all those operations when at any time you only use one of the results

TABLE 4.2 Desired calculator operations

Inputs			Operation	Sample output if A=00001111, B=00000101
x	y	z		
0	0	0	$S = A + B$	S=00010100
0	0	1	$S = A - B$	S=00001010
0	1	0	$S = A + 1$	S=00010000
0	1	1	$S = A$	S=00001111
1	0	0	$S = A \text{ AND } B$ (bitwise AND)	S=00000101
1	0	1	$S = A \text{ OR } B$ (bitwise OR)	S=00001111
1	1	0	$S = A \text{ XOR } B$ (bitwise XOR)	S=00001010
1	1	1	$S = \text{NOT } A$ (bitwise complement)	S=11110000



Big Picture – Simplified CPU

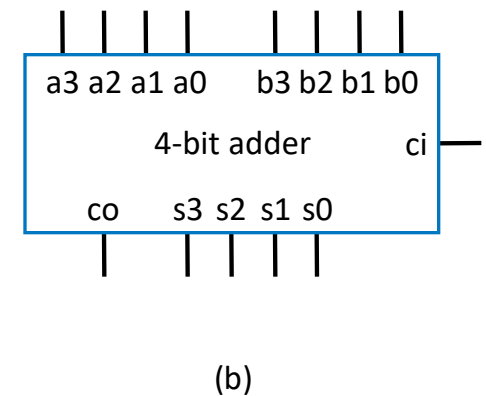
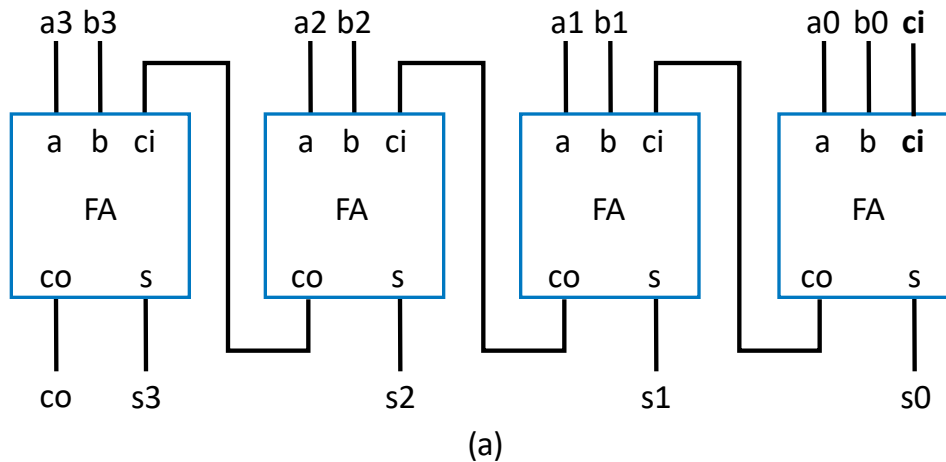


Carry-Ripple Adder

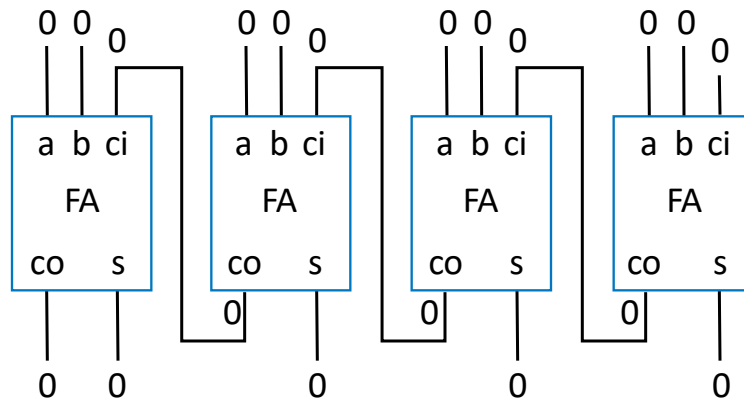
- Carry-ripple adder

- 4-bit adder: Adds two 4-bit numbers, generates 5-bit output
 - 5-bit output can be considered 4-bit “sum” plus 1-bit “carry out”
- Can easily build any size adder

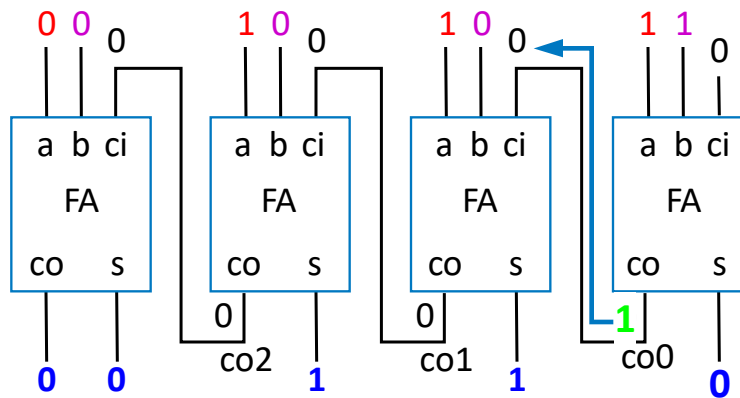
carries:	c3	c2	c1	cin	
B:	b3	b2	b1	b0	
A:	+	a3	a2	a1	a0
	cout	s3	s2	s1	s0



Carry-Ripple Adder's Behavior



Assume all inputs initially 0

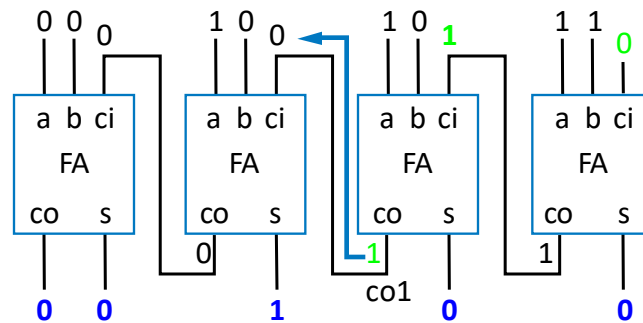


0111+0001
(answer should be 01000)

Output after 2 ns (1 FA delay)

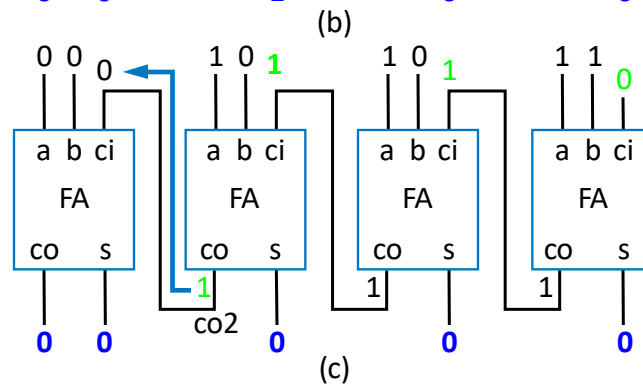
Wrong answer -- something wrong? No -- just need more time for carry to ripple through the chain of full adders.

Carry-Ripple Adder's Behavior

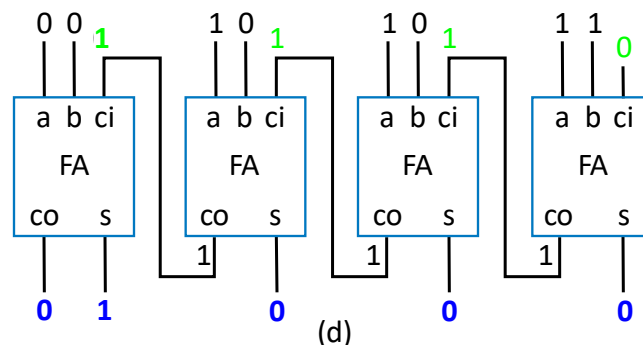


0111+0001
(answer should be 01000)

Outputs after 4ns (2 FA delays)



Outputs after 6ns (3 FA delays)

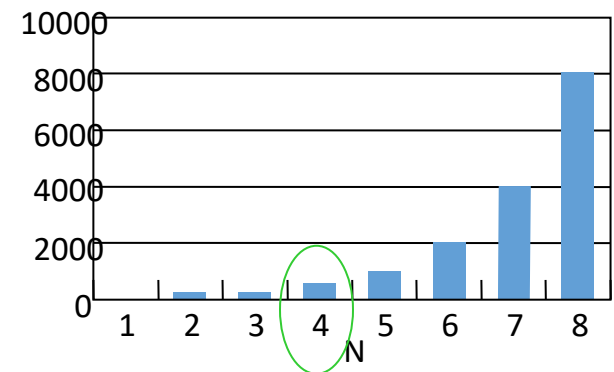
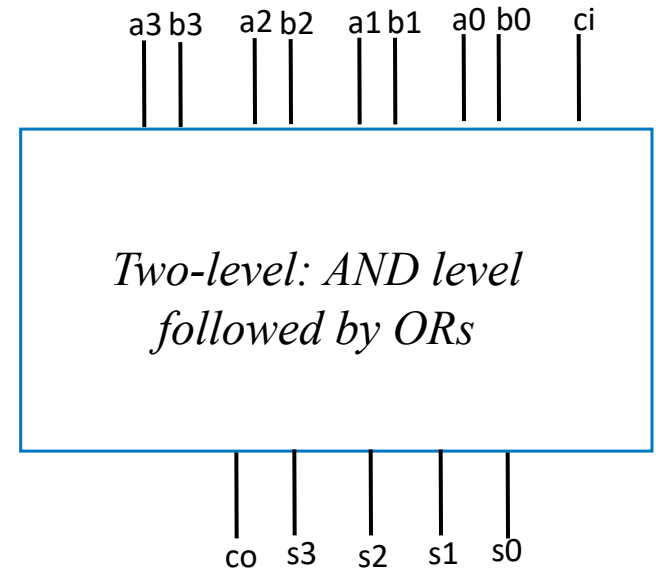


Output after 8ns (4 FA delays)

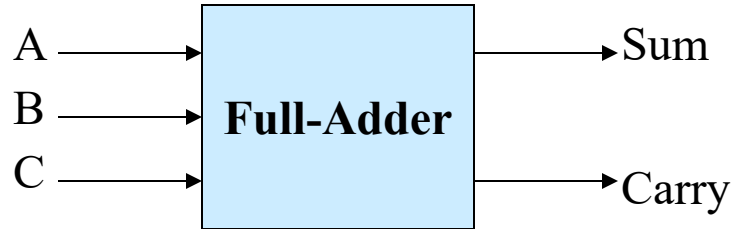
Correct answer appears after 4 FA delays

Faster Adder

- Faster adder – Use two-level combinational logic design process
 - 4-bit two-level adder is big
 - 9 input and 5 output combination circuit
 - Pro: Fast
 - 2 gate-level delays
 - Con: Large
 - Truth table would have $2^{(4+4+1)} = 512$ rows
 - Plot shows 4-bit adder would use about 500 gates



Recall Full Adder



A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned}\text{Sum} &= A'B'C + A'BC' + AB'C' + ABC \\ &= \Sigma m(1, 2, 4, 7)\end{aligned}$$

$$\begin{aligned}\text{Carry} &= A'BC + AB'C + ABC' + ABC \\ &= \Sigma m(3, 5, 6, 7)\end{aligned}$$

$$\begin{aligned}\text{Sum} &= A'B'C + A'BC' + AB'C' + ABC \\ &= A'(B'C + BC') + A(B'C' + BC) \\ &= A'(B \oplus C) + A(B \oplus C)' \\ &\quad (\text{let } B \oplus C = D) \\ &= A'D + AD' \\ &= A \oplus D \\ &= \mathbf{A \oplus B \oplus C}\end{aligned}$$

$$\begin{aligned}\text{Carry} &= A'BC + AB'C + ABC' + ABC \\ &= A'BC + AB'C + AB(C' + C) \\ &= A'BC + AB'C + AB \\ &= (A'C + A)B + A(B'C + B) \\ &= (C + A)B + A(C + B) \\ &= CB + AB + AC + AB \\ &= \mathbf{AB + AC + BC} \\ &= (A'B + AB')C + AB(C' + C) \\ &= \mathbf{(A \oplus B)C + AB}\end{aligned}$$

Faster Adder – Intuitive Attempt at “Lookahead”

- Produce carries directly

$$c_1 = a_0b_0 + a_0c_0 + b_0c_0$$

$$c_2 = a_1b_1 + a_1c_1 + b_1c_1$$

$$= a_1b_1 + a_1(a_0b_0 + a_0c_0 + b_0c_0) + b_1(a_0b_0 + a_0c_0 + b_0c_0)$$

$$c_3 = a_2b_2 + a_2c_2 + b_2c_2$$

$$= \dots\dots \text{(replace } c_2 \text{)}$$

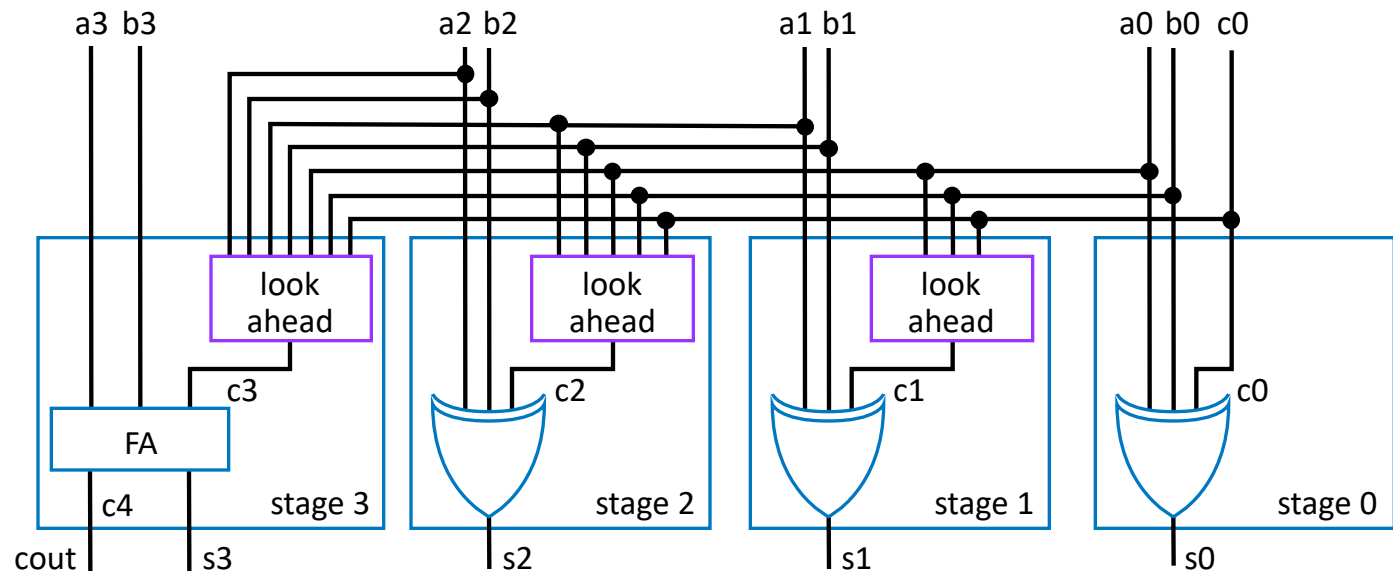
$$c_4 = a_3b_3 + a_3c_3 + b_3c_3$$

$$= \dots\dots \text{(replace } c_3 \text{)}$$

- Carry outputs of all FAs are represented with $a_0, a_1, a_2, a_3,$
 $b_0, b_1, b_2, b_3,$ and c_0

Faster Adder – Intuitive Attempt at “Lookahead”

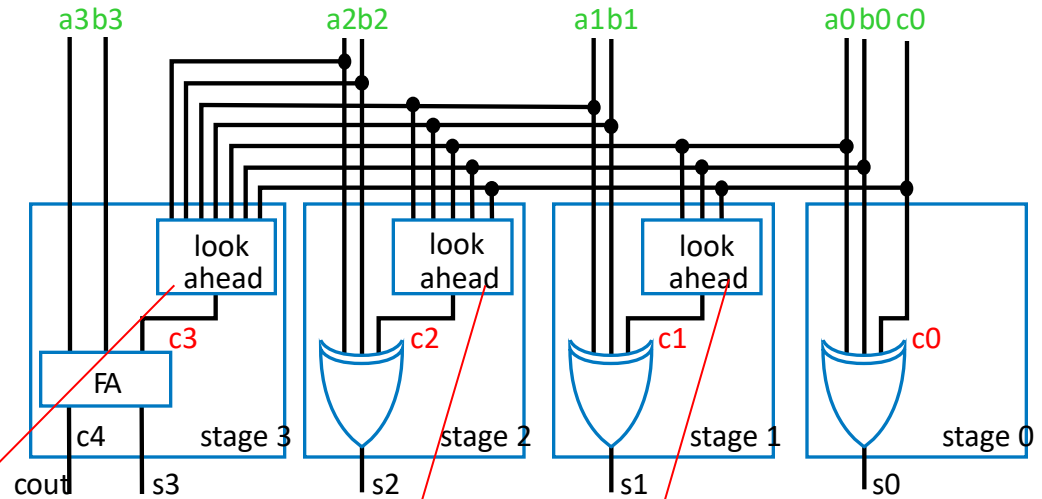
- Idea:
 - don't wait for carry to ripple through the chain, but rather **directly compute carries** from inputs
- Called carry “**lookahead**” adder



Notice – no rippling of carry

Faster Adder – Intuitive Attempt at “Lookahead”

- Carry lookahead logic
 - No waiting for ripple
 - 2-layer SOP logic
- Problem
 - Equations get too big
 - Not efficient
 - Need a better form of lookahead



$$c1 = b0c0 + a0c0 + a0b0$$

$$c2 = b1b0c0 + b1a0c0 + b1a0b0 + a1b0c0 + a1a0c0 + a1a0b0 + a1b1$$

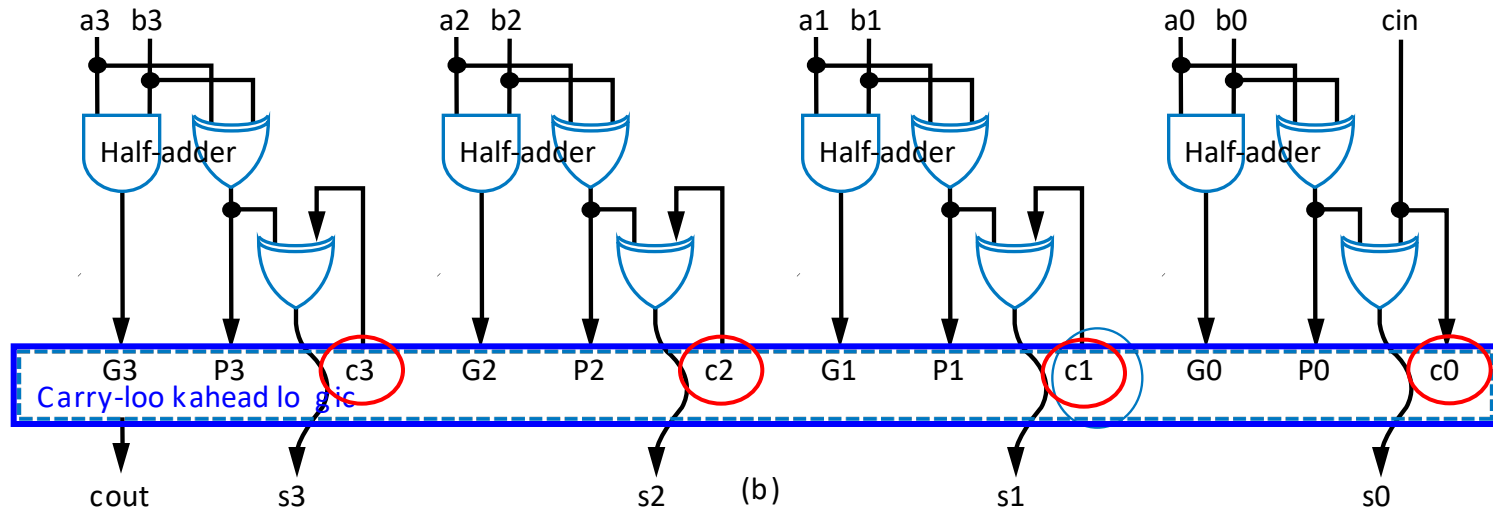
$$c3 = b2b1b0c0 + b2b1a0c0 + b2b1a0b0 + b2a1b0c0 + b2a1a0c0 + b2a1a0b0 + b2a1b1 + a2b1b0c0 + a2b1a0c0 + a2b1a0b0 + a2a1b0c0 + a2a1a0c0 + a2a1a0b0 + a2a1b1 + a2b2$$

Huge number of gates

Better Form of Lookahead

- Recall in Full Adder, another equation for carry
Carry = $ab + (a \oplus b)c$
Logic statement: Carry out is **generated** when $a=1$ and $b=1$, or **propagated** from carry-in when $a \neq b$.
- Define two terms
 - **Propagate**: $P = a \oplus b$
 - **Generate**: $G = ab$
- Compute lookahead carries from P and G terms, *not from external inputs*
 - $C_{out} = G + Pc$
 - $c_1 = a_0b_0 + (a_0 \oplus b_0)c_0 = G_0 + P_0c_0$
 - $c_2 = a_1b_1 + (a_1 \oplus b_1)c_1 = G_1 + P_1c_1$
 - $c_3 = a_2b_2 + (a_2 \oplus b_2)c_2 = G_2 + P_2c_2$

Better Form of Lookahead



- With P & G , the carry lookahead equations are much simpler

– Equations before plugging in

- $c_1 = G_0 + P_0c_0$
- $c_2 = G_1 + P_1c_1$
- $c_3 = G_2 + P_2c_2$
- $c_{out} = G_3 + P_3c_3$

After plugging in:

$$c_1 = G_0 + P_0c_0$$

$$c_2 = G_1 + P_1c_1 = G_1 + P_1(G_0 + P_0c_0)$$

$$c_2 = G_1 + P_1G_0 + P_1P_0c_0$$

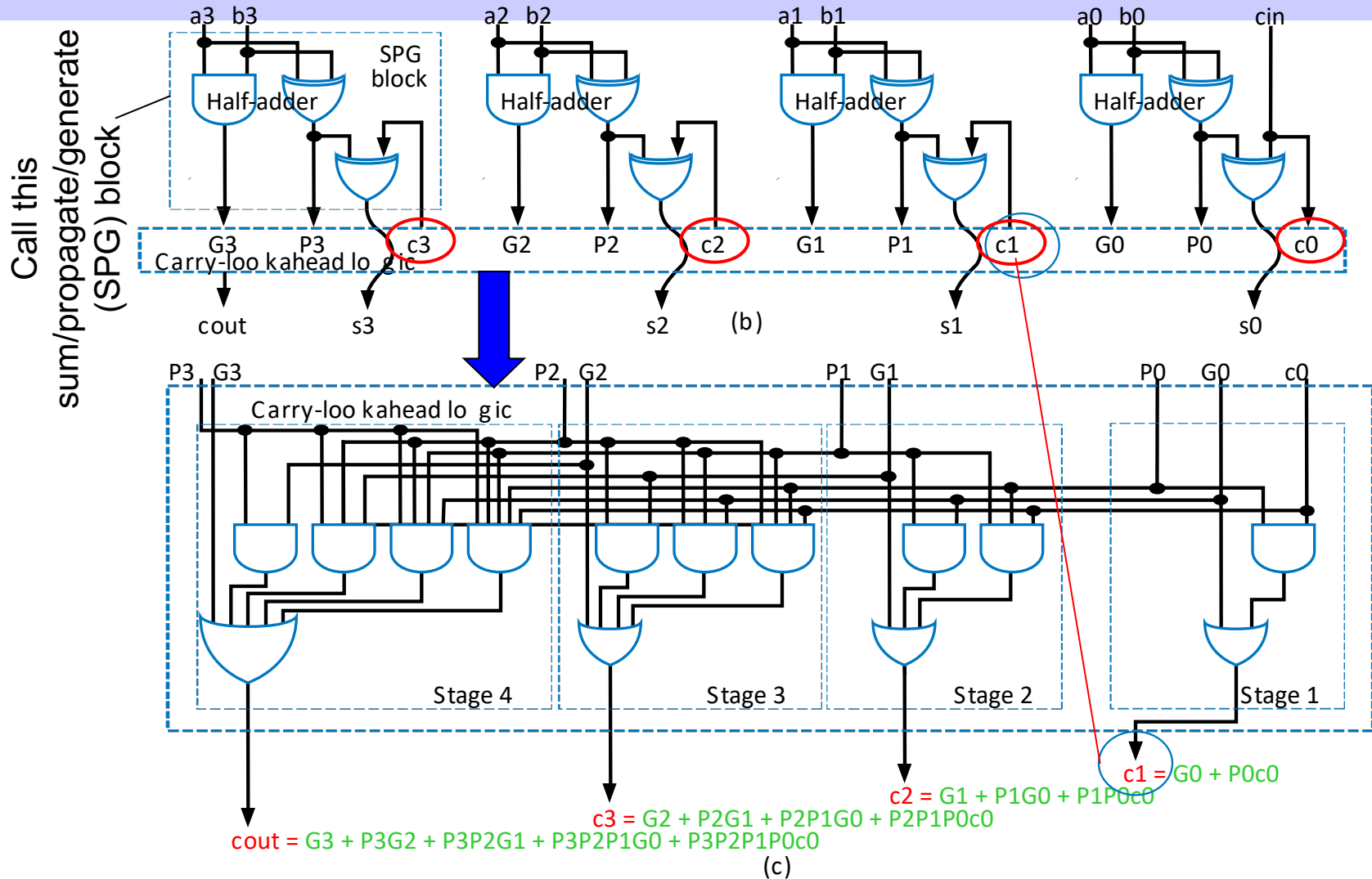
$$c_3 = G_2 + P_2c_2 = G_2 + P_2(G_1 + P_1G_0 + P_1P_0c_0)$$

$$c_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0c_0$$

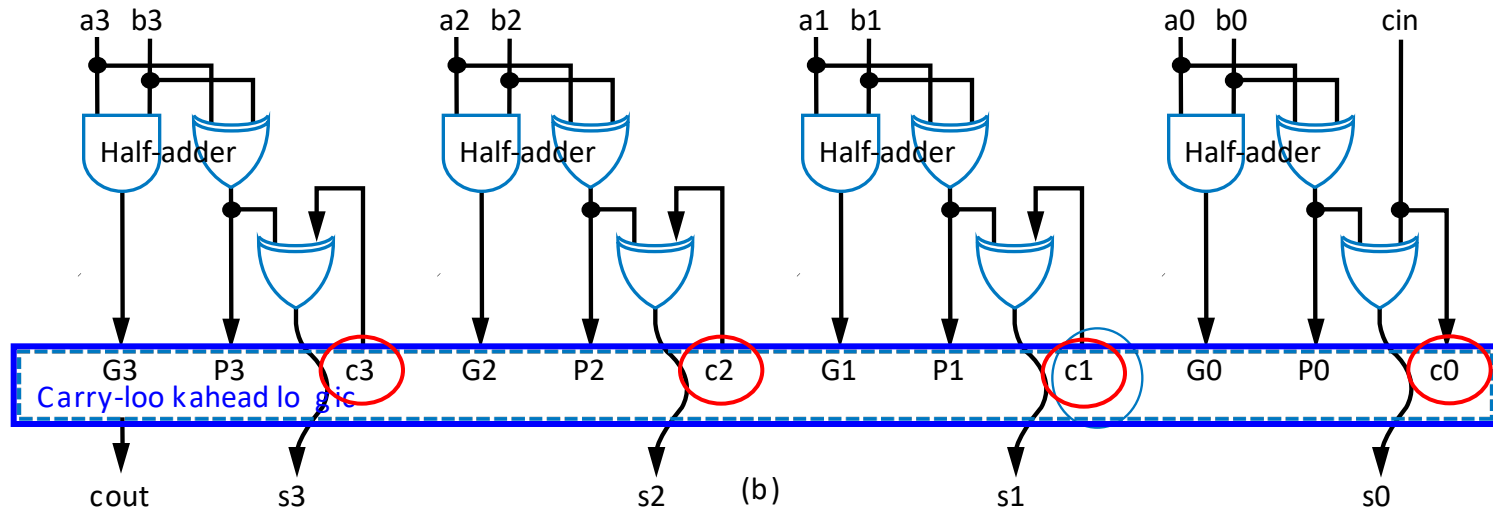
$$c_{out} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0c_0$$

Much simpler than the intuitive lookahead 15

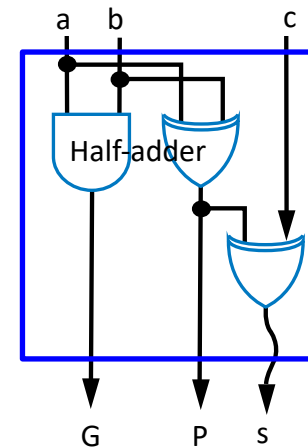
Better Form of Lookahead (cont.)



Better Form of Lookahead (cont.)

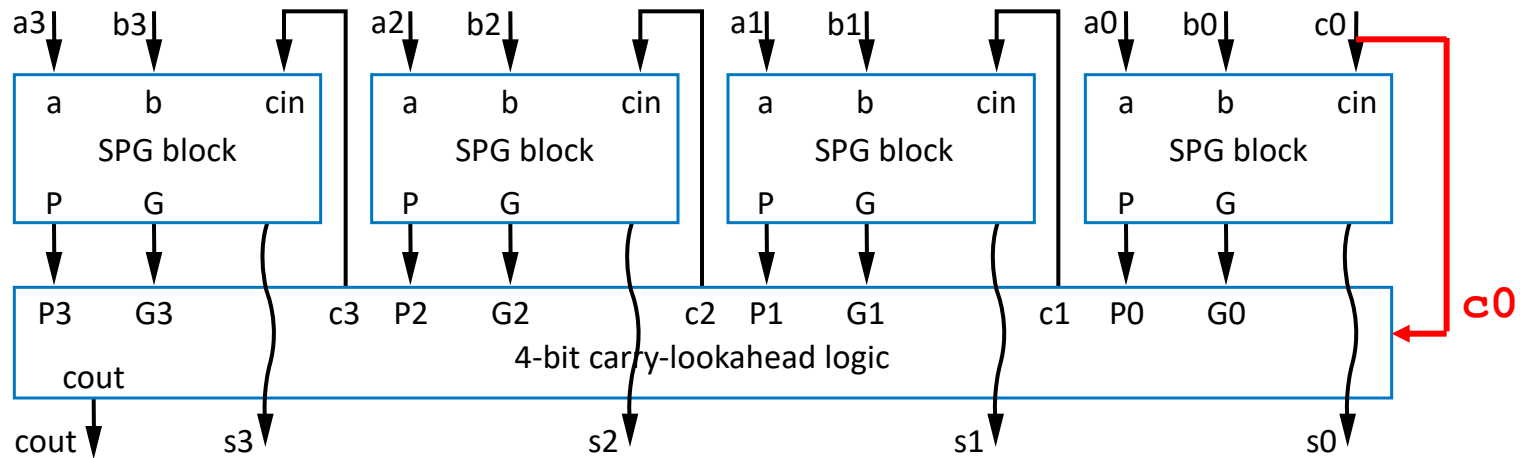


- With P & G , sum outputs are
 - $s_0 = P_0 \oplus c_{in}$
 - $s_1 = P_1 \oplus c_1$
 - $s_2 = P_2 \oplus c_2$
 - $s_3 = P_3 \oplus c_3$



SPG block

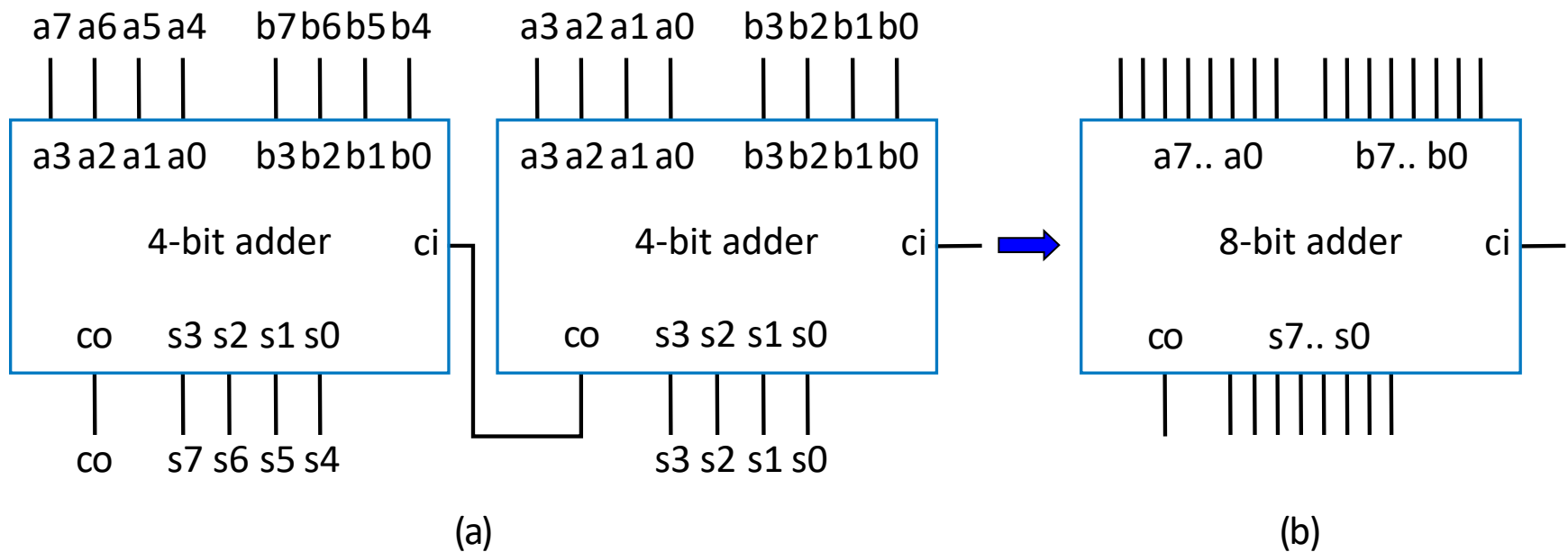
Carry-Lookahead Adder -- High-Level View



- Fast -- only **4 gate level delays**
 - Each stage has SPG block with 2 gate levels
 - Carry-lookahead logic quickly computes the carry from the propagate and generate bits using 2 gate levels inside
- Reasonable number of gates
- Nice balance between intuitive lookahead and pure combinational logic

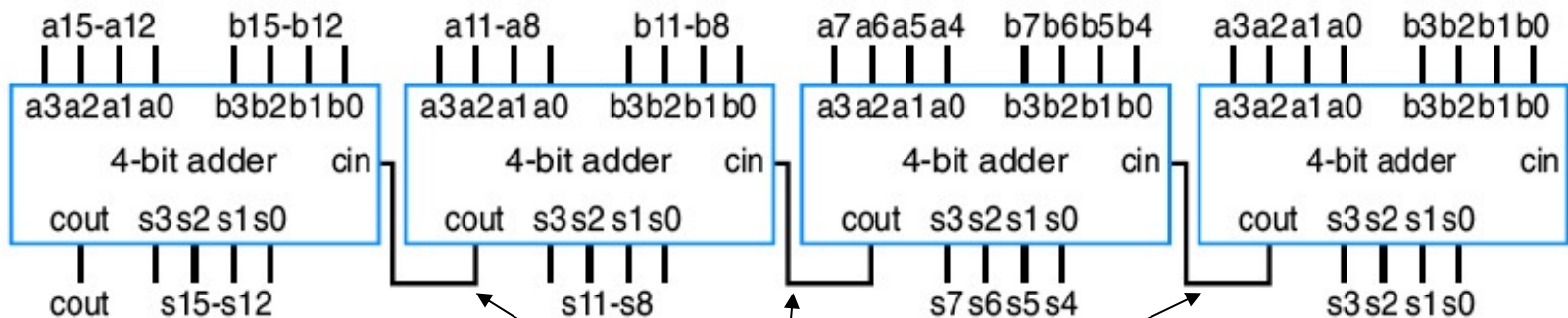
Cascading Adders

- Example: construct an 8-bit adder with two 4-bit adders



Cascading Adders

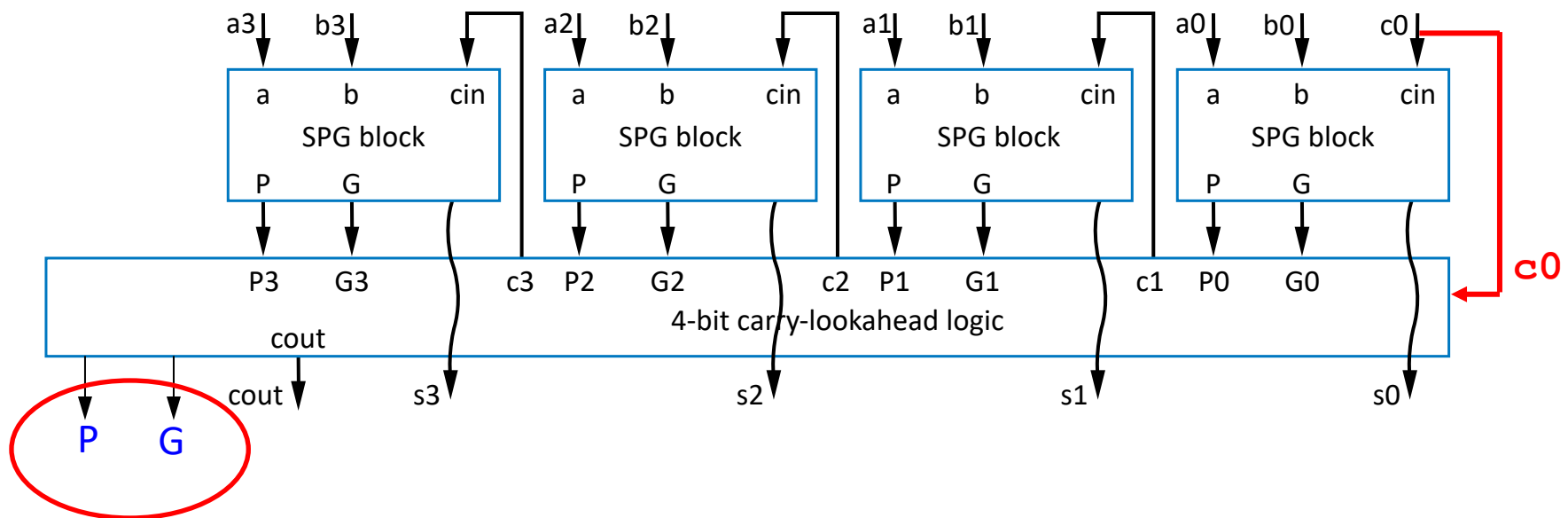
- Example: construct an 16-bit adder with four 4-bit adders



Again, carries cause propagation delay

Carry-Lookahead Adder -- High-Level View

- Adding two more outputs: P, G (group Propagate and group Generate)

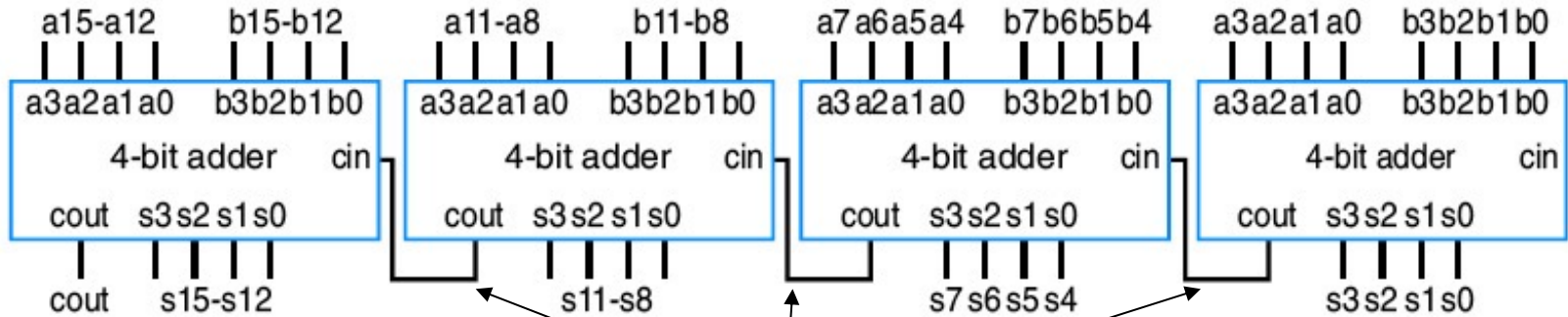


$P = P_3P_2P_1P_0$ (P is propagated through stages)

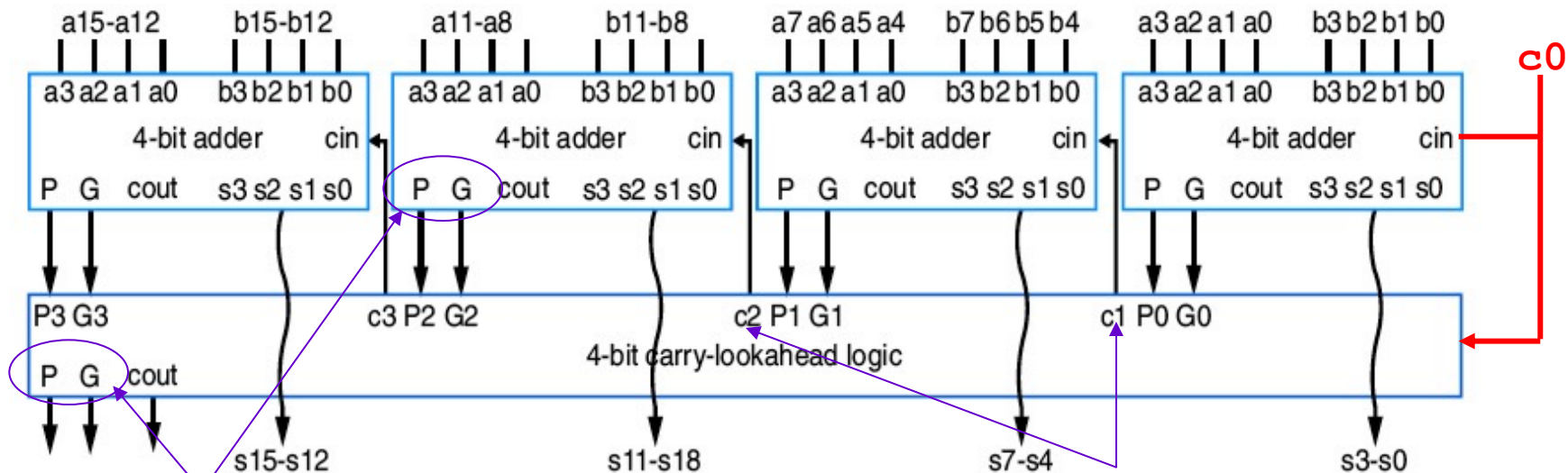
$G = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$ (G is generated in a stage and then propagated through)

Cascading Adders to CLA Addres

- Example: construct an 16-bit adder with four 4-bit adders



Again, carries cause propagation delay



$$P = P_3P_2P_1P_0$$

$$G = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

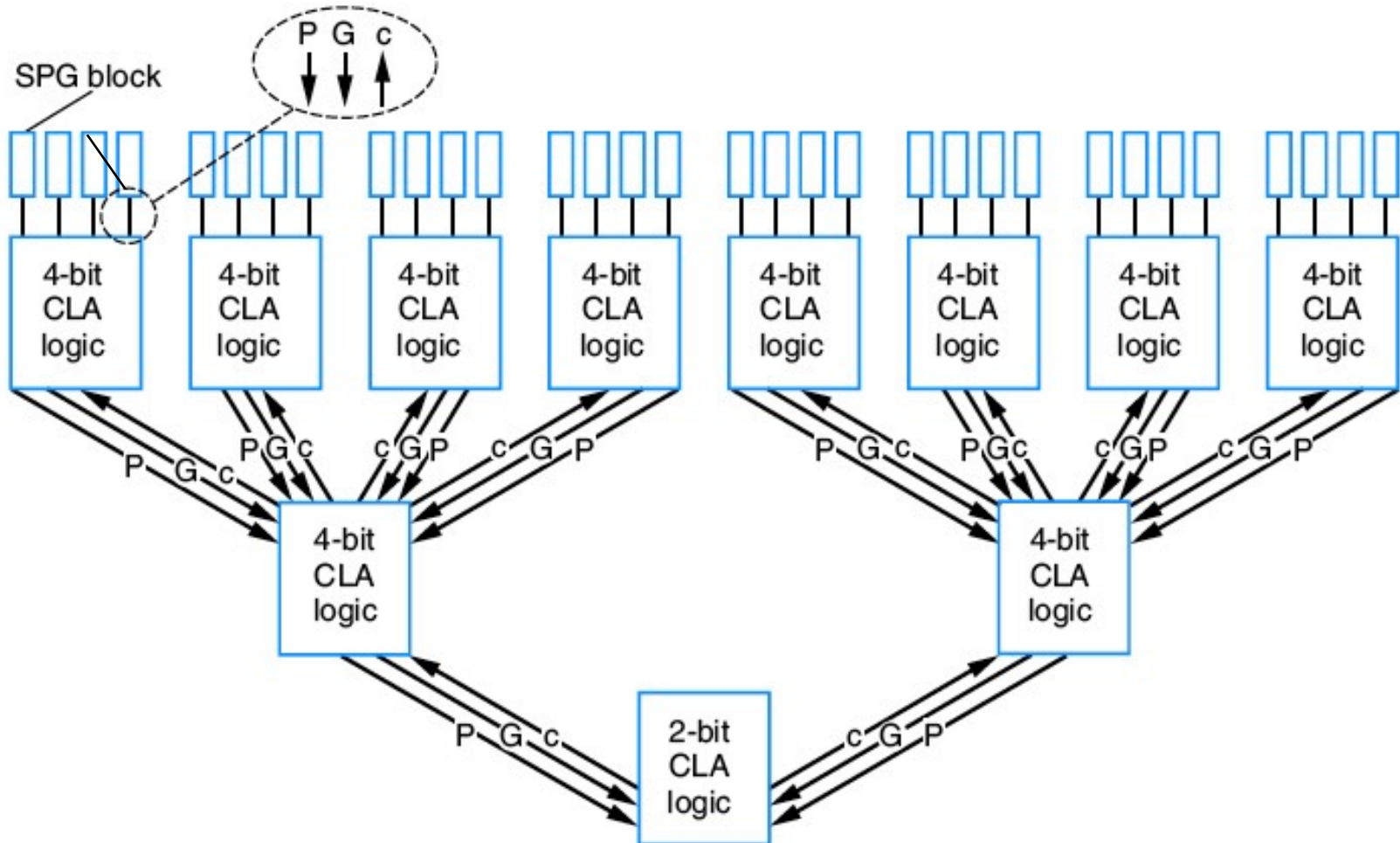
$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$$

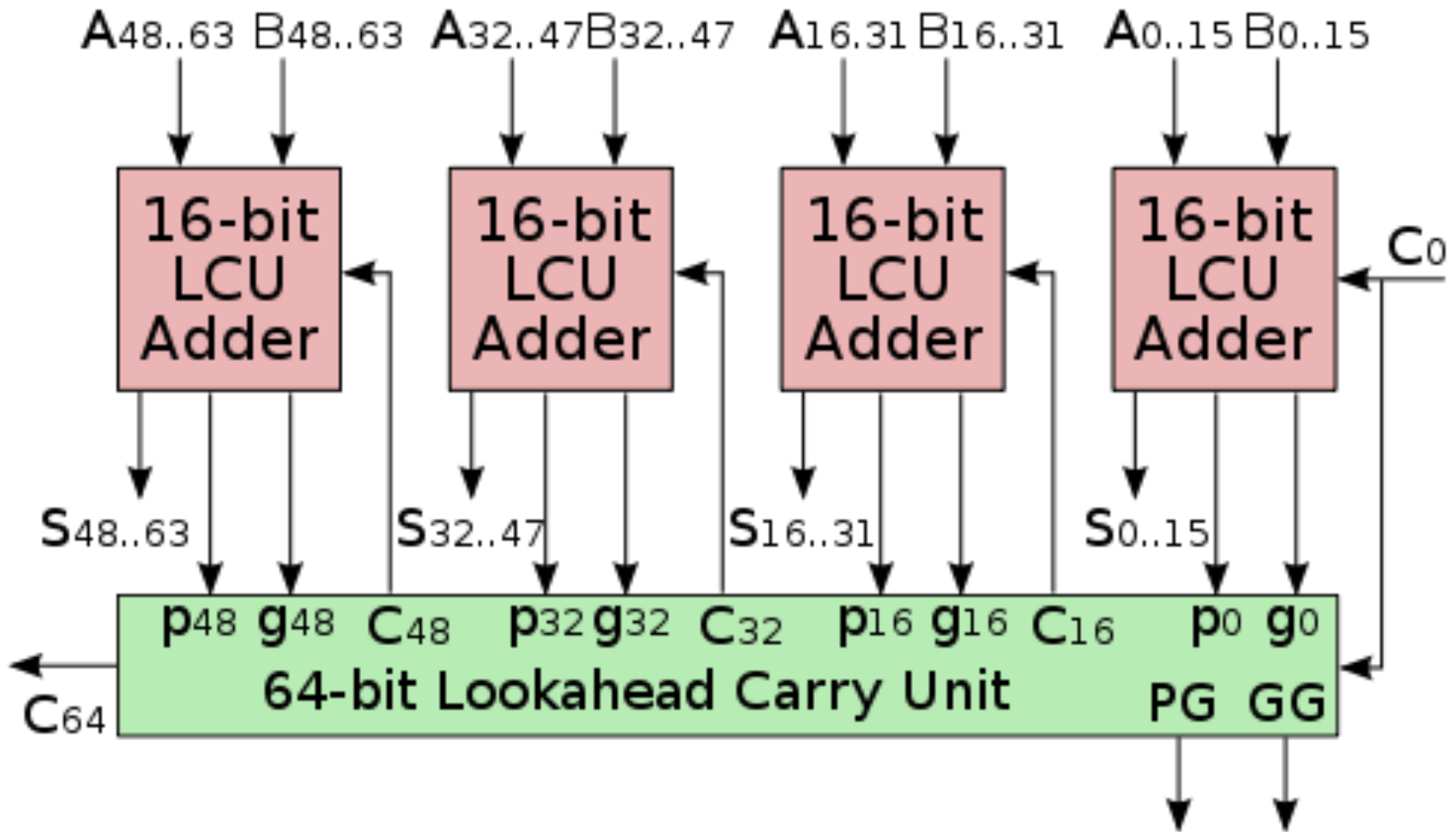
$$\dots\dots$$

CLA Adders

- Multi-level CLA structure

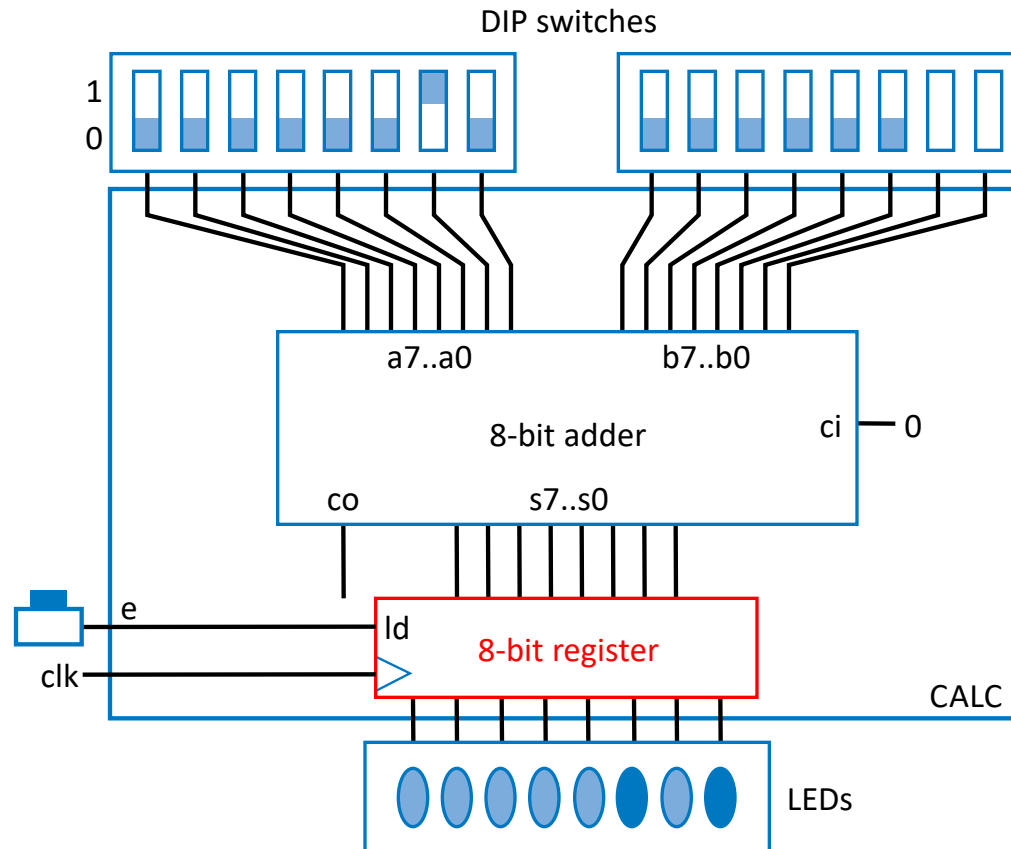


CLA Adders



Adder Example: DIP-Switch-Based Adding Calculator

- To prevent spurious values from appearing at output, can place register at output



Multiplier

- Can build multiplier that mimics multiplication by hand
 - Notice that multiplying multiplicand by 1 is same as ANDing with 1

0110	(the top number is called the <i>multiplicand</i>)
0011	(the bottom number is called the <i>multiplier</i>)
----	(each row below is called a <i>partial product</i>)
0110	(because the rightmost bit of the multiplier is 1, and $0110 * 1 = 0110$)
0110	(because the second bit of the multiplier is 1, and $0110 * 1 = 0110$)
0000	(because the third bit of the multiplier is 0, and $0110 * 0 = 0000$)
+0000	(because the leftmost bit of the multiplier is 0, and $0110 * 0 = 0000$)

00010010	(the <i>product</i> is the sum of all the partial products: 18, which is $6 * 3$)

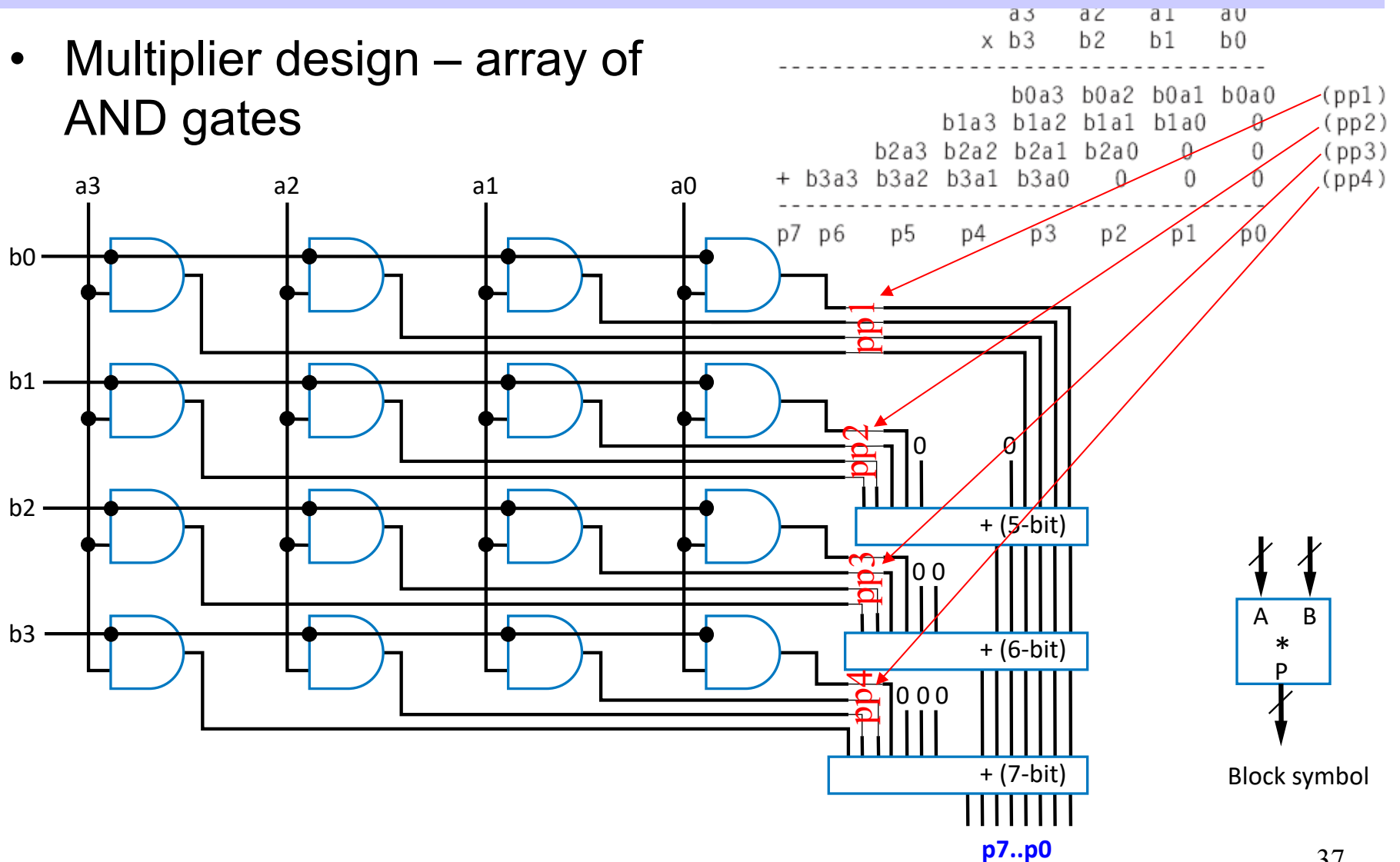
Multiplier – Array Style

- Generalized representation of multiplication by hand

$$\begin{array}{rcccccccc}
 & & & & a_3 & a_2 & a_1 & a_0 \\
 & & & x & b_3 & b_2 & b_1 & b_0 \\
 \hline
 & & & & & b_0a_3 & b_0a_2 & b_0a_1 & b_0a_0 & (pp1) \\
 & & & & & b_1a_3 & b_1a_2 & b_1a_1 & b_1a_0 & 0 & (pp2) \\
 & & & & & b_2a_3 & b_2a_2 & b_2a_1 & b_2a_0 & 0 & 0 & (pp3) \\
 + & b_3a_3 & b_3a_2 & b_3a_1 & b_3a_0 & 0 & 0 & 0 & 0 & 0 & (pp4) \\
 \hline
 p_7 & p_6 & p_5 & p_4 & p_3 & p_2 & p_1 & p_0
 \end{array}$$

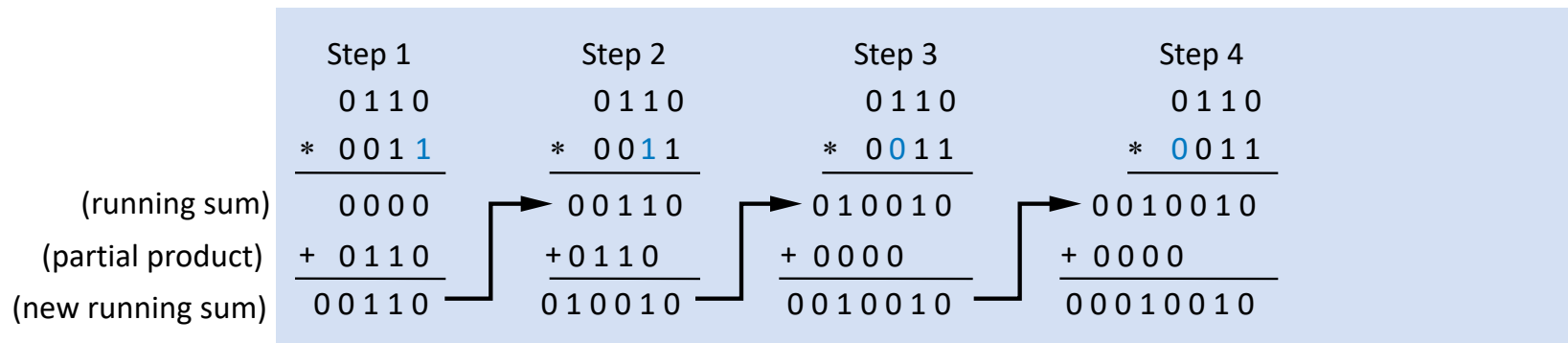
Multiplier – Array Style

- Multiplier design – array of AND gates

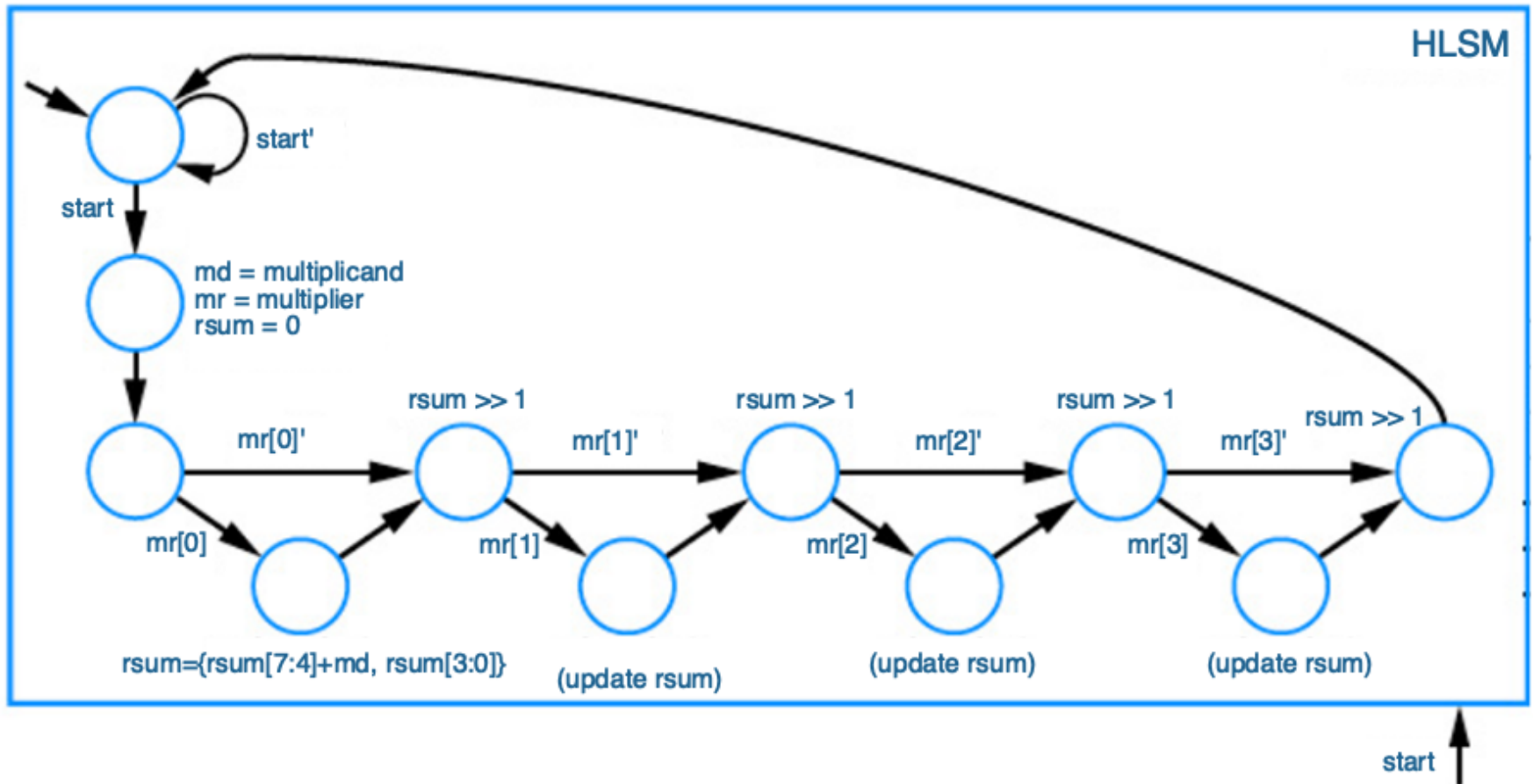


Smaller Multiplier -- Sequential (Add-and-Shift) Style

- Add-and-Shift
 - Don't compute all partial products simultaneously
 - Rather, compute one at a time (similar to by hand), maintain a *running sum*

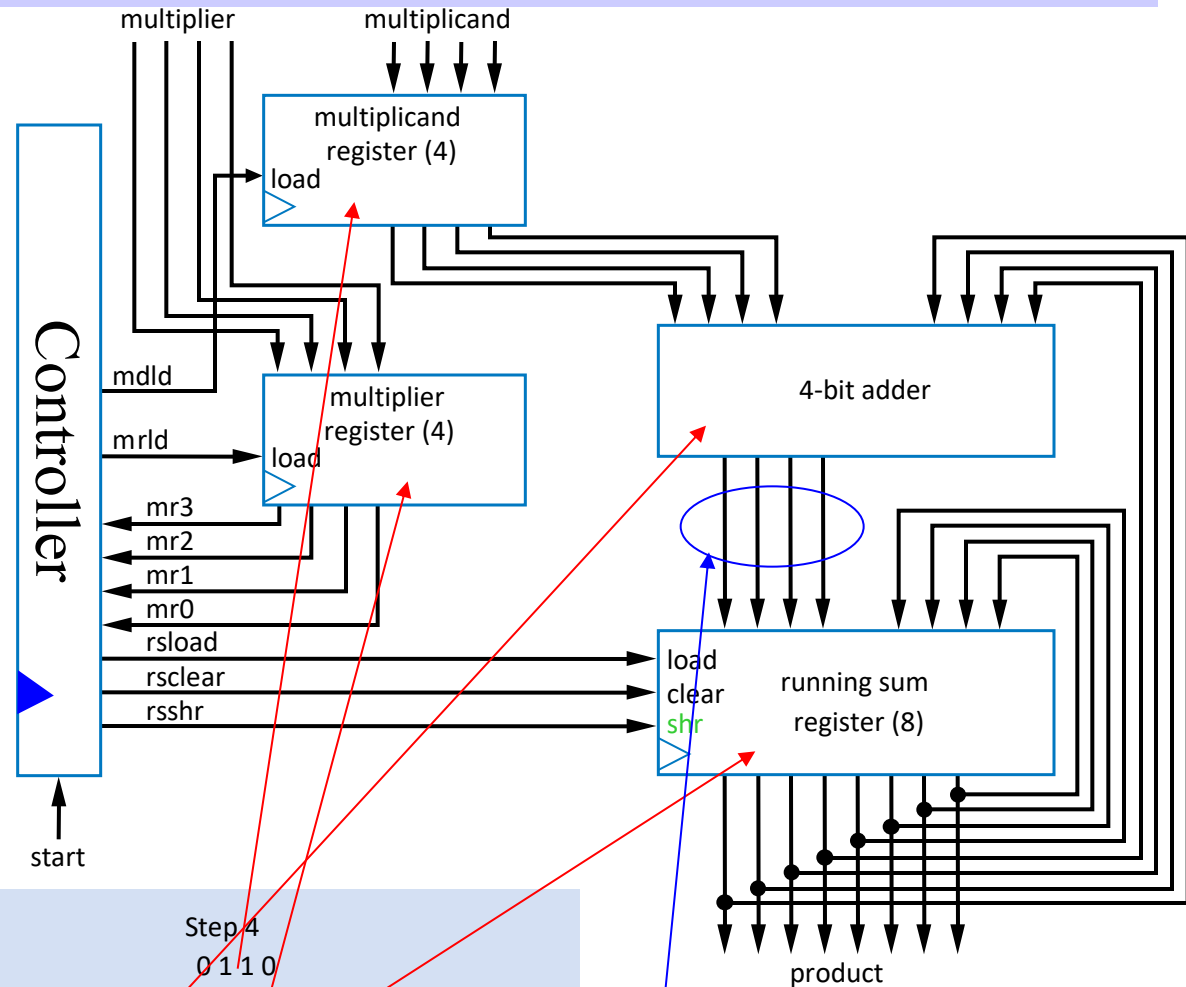


Smaller Multiplier – HLSM



Smaller Multiplier – Datapath

- Design circuit that computes one partial product at a time, adds to running sum
 - Note that **shifting** running sum right (relative to partial product) after each step ensures partial product added to correct running sum bits

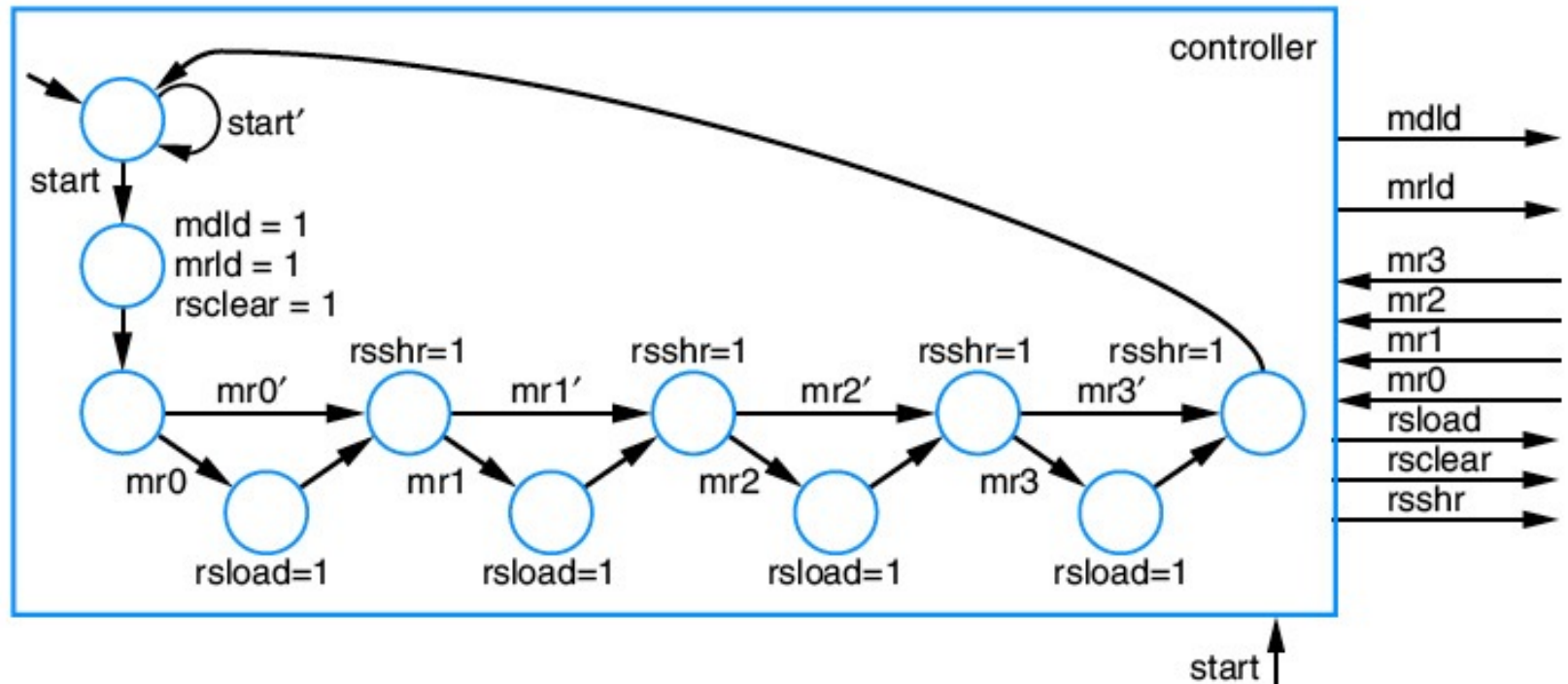


Step 1	Step 2	Step 3	Step 4	
0110	0110	0110	0110	
+ 0011	+ 0011	+ 0011	+ 0011	
0000	00110	010010	0010010	(running sum)
+ 0110	+ 0110	+ 0000	+ 0000	(partial product)
00110	010010	0010010	00010010	(new running sum)

What about carry?

What if 2's complement?

Smaller Multiplier – Controller Design



Summary

- Carry lookahead concept
 - Why?
 - How?
 - How much improvement in delay and size?
- Learn functions of arithmetic building blocks (datapath components) to be used in a digital system
- Multiplier may be designed as a combinational circuit or as a digital system (controller + datapath)
 - How?
 - Why?